subfolders within the JAR. We could expand standard JAR artifacts and put each folder on a different container image level. Starting with version 2.3, Spring Boot made that even more efficient by introducing a new way of packaging applications as JAR artifacts: the layered-JAR mode. And since Spring Boot 2.4, that's been the default mode, so you don't need any extra configuration to use the new functionality.

Applications packaged using the *layered-JAR mode* are made up of layers, similar to how container images work. This new feature is excellent for building more efficient images. When using the new JAR packaging, we can expand the JAR artifact and then create a different image layer for each JAR layer. The goal is to have your own classes (which change more frequently) on a separate layer from the project dependencies (which change less frequently).

By default, Spring Boot applications are packaged as JAR artifacts made up of the following layers, starting from the lowest:

- `dependencies`—For all the main dependencies added to the project
- `spring-boot-loader`—For the classes used by the Spring Boot loader component
- `snapshot-dependencies`—For all the snapshot dependencies
- `application`—For your application classes and resources

If you consider the previous scenario where you added a new REST endpoint to an existing application, only the `application` layer must be built when you containerize it. Moreover, when you upgrade the application in production, only that new layer must be downloaded to the